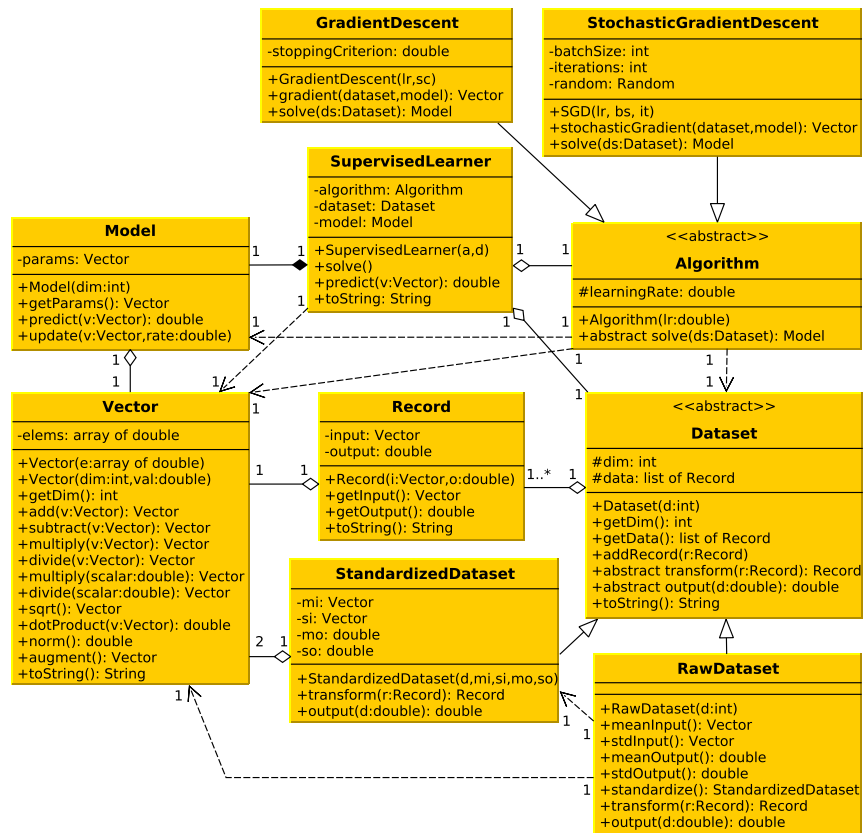


Lab 4: Stochastic gradient descent

24292-Object Oriented Programming

1 Introduction

The aim of this lab is to implement linear regression with stochastic gradient descent and standardized datasets using the design from Seminar 4. The session is mandatory and you have to deliver the source code of the Java project as well as a document that outlines the solution of the problem, using the guidelines from Lab 1. As usual, a sample solution appears below.



2 Existing classes

Start by copying the existing classes of the project to a new directory. The classes `Vector`, `Record` and `Model` should be identical to before. The class `SupervisedLearner` is almost identical, but as explained below it is necessary to modify the method `predict`. As usual, create a class `TestLearner` with a main method where the different class definitions are thoroughly tested.

3 Datasets

Compared to the previous lab, the design of the classes representing datasets has changed significantly, making it necessary to reimplement the Java code for these classes. Modifying the code of an existing set of classes is called *refactoring*. Concretely, you have to make the following changes with respect to the previous implementation:

- The class `Dataset` is now abstract, and no longer contains methods for computing the mean and standard deviation of inputs and outputs, nor the method `standardize`.
- Instead, these methods are part of the new class `RawDataset` that inherits from `Dataset`.
- The class `Dataset` contains abstract methods `transform` for transforming a record and `output` for translating an output using the reverse transformation. All datasets should be capable of performing these operations.
- Both `StandardizedDataset` and `RawDataset` need to override these two abstract methods. The implementation details for each of these classes are explained in the handout of Seminar 4.

4 Stochastic gradient descent

The next step is to implement the classes representing algorithms. Concretely, the `Algorithm` class is now abstract, and contains an attribute representing the learning rate which is shared by all algorithms. The class also contains an abstract method `solve` since all supervised learning algorithms should be capable of learning and returning a model given a dataset.

The class `GradientDescent` contains the same methods as the former class `Algorithm`, as well as the attribute for the stopping criterion which is specific for this class. These methods can be implemented the same way as before, so you can basically just copy the code from the old class `Algorithm` of the previous lab.

The class `StochasticGradientDescent` is new and should contain attributes for the batch size and number of iterations, as well as the code for performing stochastic gradient descent. To do so, it is useful to implement a method

`stochasticGradient` that samples a minibatch of records and computes a stochastic gradient using Equation (1) from Seminar 4. To sample a subset of distinct indices, you can use the following method from the existing Java class `Random`:

```
Random random = new Random();  
int[] r = random.ints(0, bound).distinct().limit(copies).toArray();
```

The array `r` now contains a number `copies` of distinct integers between 0 and `bound-1`, where `copies` should be less than or equal to `bound`. You can directly use the integers of `r` to access the appropriate elements of the dataset. This way it is not necessary to actually create a separate collection that stores the minibatch. Finally, the method `solve` is where the algorithm iteratively applies stochastic gradient descent a number of times determined by its attribute.

5 The predict method

To handle standardized datasets, it is necessary to modify the `predict` method of `SupervisedLearner`. Concretely, when receiving an input as argument, the input has to first be transformed using the `transform` method of `Dataset`. Since `transform` expects an instance of `Record` as argument, you can either create a dummy instance of `Record` using the given input, or implement a separate `transform` method for vectors in `Dataset`.

Once you have transformed the input, you can now augment it and use the `Model` class to obtain a prediction. However, for standardized datasets this prediction does not belong to the original space, so we have to translate it back using the `output` method of `Dataset` before returning it.

6 Practical experimentation

The last step of the lab is to experiment with stochastic gradient descent in the main method of `TestLearner`. Concretely, to make stochastic gradient descent work well, it is necessary to tune the learning rate, the batch size and the number of iterations. Since a stochastic gradient is less precise than a full gradient, typically the learning rate has to be slightly smaller than that of gradient descent. Usually stochastic gradient descent is much faster than gradient descent, but this is hard to reproduce in the lab since you need a dataset with a lot of records (in the millions). At the very least the resulting model output by stochastic gradient descent should be similar to that output by gradient descent. Your code should also work for both raw datasets and standardized datasets, with similar results.

7 Optional: logistic regression

Logistic regression is a different supervised learning setting in which the outputs are *probabilities*. Hence all records (x_i, y_i) of a dataset $\mathcal{D}$ should satisfy $y_i \in$

$[0, 1]$. In logistic regression, the learned function f takes the form of a sigmoid given by

$$f(x) = \frac{1}{1 + e^{-\theta \cdot \bar{x}}},$$

where $\bar{x}$ is the augmented input vector associated with x and θ is again a $(d + 1)$ -dimensional parameter vector. For a particular choice of loss function, the gradient for an individual record (x_i, y_i) is given by

$$\bar{x}_i \left(\frac{1}{1 + e^{-\theta \cdot \bar{x}_i}} - y_i \right).$$

If we compare with Equation (2) for linear regression from Seminar 3, the gradient for an individual record has exactly the same form $\bar{x}_i(f(x_i) - y_i)$, and the only difference is in the definition of the function f .

The optional task consists in modifying the code such that it is capable of solving logistic regression problems. Gradient descent and stochastic gradient descent work exactly the same as before: compute the average gradient of all records or a minibatch of records, and iteratively update the model θ using Equation (3) of Seminar 3. To make the code as modular as possible, we recommend creating an abstract class `PredictorFunction` or similar with subclasses corresponding to the concrete functions for linear regression and logistic regression. The classes `SupervisedLearner` and `Algorithm` need instances of `PredictorFunction`, and the code for computing the gradient or stochastic gradient should call a method of `PredictorFunction` to compute the correct gradient associated with an individual record. This should only involve changing a single line of code in the methods `gradient` and `stochasticGradient`. Note that the class `SupervisedLearner` also has to apply the correct predictor function when making a prediction. Recall that to test your code, you need to create a dataset of records whose outputs are probabilities.

8 Submission

The source code and documentation should all be uploaded to a directory **Lab4** of your Git repository **prior to the next lab**. Please make sure that the directory name is *exactly* **Lab4** (not **Lab_4**, **Lab-4**, **LAB4**, **lab4** etc.), and that **Lab4** is located in the **root of the Git repository**. Failure to comply with these rules will result in a grade deduction.